

Prueba Técnica para **Desarrollador Senior Laravel**

Esta prueba técnica está diseñada para evaluar tus habilidades como desarrollador(a) **Full-Stack** con tecnologías **Laravel**, **React** y **TypeScript**, tomando en cuenta un entorno moderno de desarrollo en el que puedes hacer uso de herramientas de **Inteligencia Artificial (IA)** como apoyo, siempre y cuando sepas adaptar y justificar su uso.

El reto consiste en construir un módulo de **sistema de mensajería tipo "inbox"**, como el que encontrarías en plataformas de soporte o correo electrónico interno. El sistema debe permitir listar conversaciones, ver mensajes dentro de un hilo, y responder a mensajes existentes.

Puedes elegir uno de los siguientes enfoques:

- **Full-Stack:** implementar tanto el backend en Laravel como el frontend en React.
- **Back-End:** enfocarte exclusivamente en la creación de APIs RESTful con Laravel.
- **Front-End:** construir la interfaz en React/TypeScript consumiendo APIs simuladas (mock).

El objetivo no es solo tener una aplicación funcional, sino demostrar cómo estructuras tus ideas, tomas decisiones técnicas, escribes código de calidad y pruebas lo que construyes.

El candidato podrá optar por uno de los siguientes enfoques:

- **Implementación Full-Stack** (Laravel + React/TypeScript + APIs completas)
- **Enfoque Directo a Back-End** (Laravel + diseño de APIs REST sin interfaz de usuario)
- **Enfoque Directo a Front-End** (React/TypeScript + consumo de APIs simuladas sin backend real)

Compartir la prueba técnica por GitHub
<https://github.com/OmarNullData>

1. Mockup de interfaz



2. APIs a construir (opción Full-Stack o Backend-Only)

1. Autenticación / Identidad

- `POST /api/login` → JWT
- `GET /api/user` → datos del usuario actual

2. Conversaciones

- `GET /api/threads` → lista de hilos (paginada, filtros, búsqueda)
- `GET /api/threads/{id}` → detalles del hilo + mensajes

3. Mensajes

- `POST /api/threads` → crear nuevo hilo con primer mensaje
- `POST /api/threads/{id}/messages` → enviar respuesta en hilo

4. Notificaciones (bonus)

- `GET /api/notifications` → mensajes no leídos, etc.

Formato: JSON

Autorización: Bearer token (JWT)

3. Entregables esperados

- **Diseño de la UI** (mockup en Figma, boceto en dibujo, o código de componentes React).
- **Implementación Back-End**
 - Rutas, controladores, modelos y migraciones en Laravel.
 - Validaciones y tests básicos (PHPUnit).
 - Documentación de la API (OpenAPI/Swagger o README).
- **Implementación Front-End**
 - Componentes React con TypeScript.
 - Llamadas a la API (fetch/Axios), manejo de estados (Context/Redux).
 - Estilos (CSS Modules, Tailwind, etc.) y diseño responsivo.
 - Tests básicos (Jest + React Testing Library).
- **Instrucciones de despliegue** (README): instalación, arranque de back y front, variables de entorno.



4. Reglas de evaluación

Criterio	Qué buscamos	Indicadores de uso de IA vs. aporte propio
Calidad del código	Limpieza, consistencia, comentarios claros.	Estructura lógica, no “pegado” de snippets sin adaptación.
Arquitectura y diseño	Separación de capas, patrones , escalabilidad (MVC, hooks).	Explicación en el README de decisiones de diseño.
Uso de Laravel / React TS	Uso de features idiomáticos (Eloquent, Middleware, Hooks, Context).	Código “hand-made” que aproveche el framework, no copiado.
Pruebas	Cobertura mínima de rutas/componentes clave.	Tests que cubran escenarios, no solo importación de librerías.
Documentación	README claro con pasos, descripción de endpoints, decisiones.	Documentación escrita por el candidato, no genérica.
Control de versiones	Commits frecuentes, mensajes descriptivos.	Historial coherente, no un solo commit enorme.
UX/UI y accesibilidad	Interacciones intuitivas, labels, foco.	Explicación de elecciones de UX; no solo “funciona”.
Originalidad y justificación	Notas en el README sobre retos y soluciones.	Reflexión personal y referencias a fuentes/IA usadas.

Nota sobre IA:

- Está permitido consultar ChatGPT, Copilot u otras herramientas, pero **debe** anotarse en el README qué partes fueron generadas por IA y cómo se adaptaron.
- El candidato debe justificar cada fragmento “sacado” de IA, mostrar que entendió y personalizó el código.
- Para la inclusión de grandes bloques “copy-paste” escribir los prompts utilizados en el README.



Entrega de la prueba técnica

Para completar el proceso, te pedimos que compartas el repositorio del proyecto con nuestro equipo a través de GitHub. Por favor, sigue estas indicaciones:

1. **Crea un repositorio privado** en tu cuenta de GitHub.
2. **Sube el código fuente** del proyecto de la prueba técnica, asegurándote de incluir un archivo **README.md** con:
 - Instrucciones de instalación y ejecución.
 - Descripción breve del enfoque utilizado y cualquier decisión técnica relevante.
 - Capturas de pantalla o ejemplos si lo consideras necesario.

Invita al siguiente usuario a colaborar como "Read-only" en tu repositorio:

<https://github.com/OmarNullData>

3. Puedes hacerlo desde la pestaña **Settings > Collaborators** del repositorio.

